Project Report: Deep Space Cargo Recovery

Interactive Graphics Course - Academic Year 2025/2026

Samuele Costantinopoli Matricola: 2271799
Emanuele Modjib Shirazi Matricola: 2263513
Group Name: Space Groovers
Course: Interactive Graphics **Professor:** Marco Schaerf
Institution: Sapienza University of Rome
Department of Computer, Control and Management Engineering (DIAG)

August 28, 2026

Abstract

This report presents the technical design and implementation of "Deep Space Cargo Recovery", a 3D interactive graphics application built for the Interactive Graphics course at Sapienza University of Rome. The application features a player-controlled exploration probe navigating a zero-gravity environment to recover energy crystals and deliver them to a mothership cargo bay. The report covers our hierarchical scene graph design, procedural kinematics, real-time lighting and PBR shaders, vector mechanics for solar charging, orbital obstacle mathematics, and custom collision pipelines developed with WebGL, Three.js, and Tween.js.

---

1 Introduction and Project Overview

In "Deep Space Cargo Recovery", the user pilots a robotic probe through deep space to retrieve scattered energy crystals and deposit them into the cargo bay of a mothership.

The primary goal was to construct a real-time 3D simulation using WebGL and Three.js while addressing all foundational topics of the course:

- A hierarchical probe model capable of mechanical joint rotations and panel deployment.
- Programmatic, keyframe-free animations computed using procedural easing functions.
- Real-time vector mechanics for solar alignment, orbital physics, and battery drainage.
- PBR materials, custom camera perspectives, and multi-tier spatial collision handling.

2 Environment and Libraries Used

The project relies on standard, lightweight web graphics tools:

- **Three.js:** Manages scene graph hierarchies, geometry buffers, PBR shader pipelines, matrix concatenations, and WebGL rendering passes.
- **Tween.js:** Handles smooth interpolation curves for mechanical joints and solar wings without relying on imported pre-baked animations.

- **lil-gui:** Integrates an interactive debug menu to modify light positions, color intensities, camera matrices, and probe states at runtime.
- **HTML5 / CSS3 (Glassmorphism HUD):** Provides telemetry panels, live energy bars, emergency low-battery overlays, and game-mode selection screen.

3 Project Access and Execution Guide

The game is deployed and immediately playable online via GitHub Pages:
https://sapienzainteractivegraphicscourse.github.io/final-project-space_groovers/

3.1 Local Installation and Setup

To run the project locally on your machine, follow these steps:

1. **Clone the repository and enter the directory:**

```
git clone https://github.com/SapienzaInteractiveGraphicsCourse/
    final-project-space_groovers
cd final-project-space_groovers
```

2. **Install project dependencies:**

```
npm install
```

3. **Start the local development server:**

```
npm run dev
```

4. **Launch in browser:** Open the local address provided in the terminal (typically `http://localhost:5173`) using any modern WebGL-compatible browser.

4 Game Architecture and Modes

4.1 Main Screen

Right after clicking the link, this is how the screen presents itself:

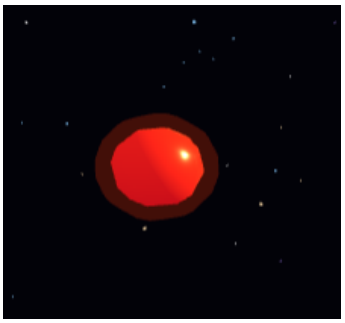
Figure 1: Main Menu Display.

4.2 Pre-Mission Material Customization

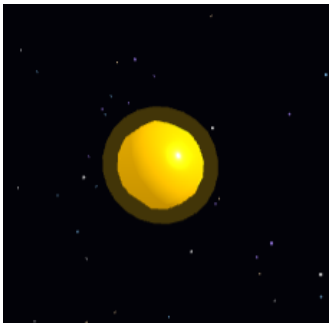
Before launching the flight simulation, the start overlay provides an interactive customization interface allowing the user to select visual themes:

- **Crystal Energy Themes:** Dynamically alters the target crystals' base color, emissive hex value, and outer glow shell hue (e.g., Cyan Plasma, Hyperion Amber, Gold Nebula). Selecting a theme immediately updates the entity spawn buffers.

(a) Cyan crystal



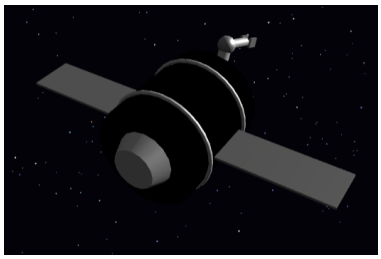
(b) Amber crystal



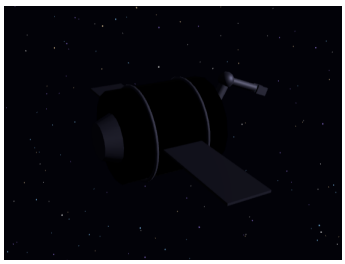
(c) Gold crystal

Figure 2: Comparison of the three available crystal textures.

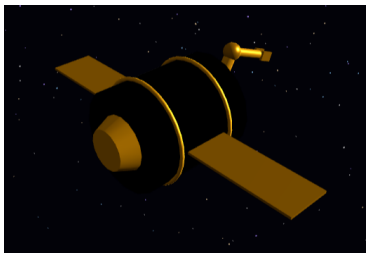
- **Ship Hull Finishes:** Adjusts the probe's primary diffuse color and surface roughness (e.g., Classic Titanium, Matte Stealth, Gold Plating) in real time.



(a) Silver probe



(b) Dark probe



(c) Gold probe

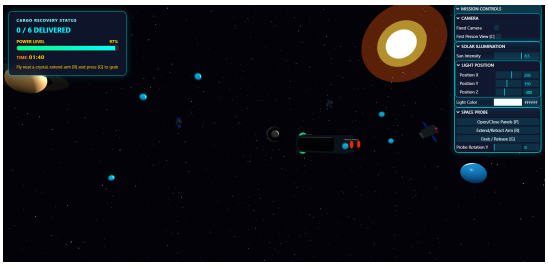
Figure 3: Comparison of the three available probe textures.

4.3 Modes and Difficulties

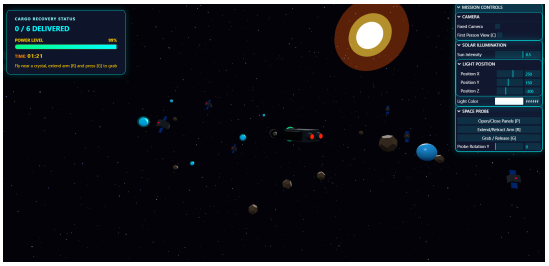
The application supports three gameplay configurations to accommodate different interaction complexities:

- **Testing the Probe:** A free-flight sandbox mode with baseline energy drain, no time limits, and an obstacle-free corridor designed to let the player learn the 6-DOF controls.
- **Running out of Time:** A timed mission with a 120 s countdown and static satellite debris placed across the map that causes instant hull failure on collision.
- **Spinning out of Space:** A hardcore scenario starting with a 90 s timer, dynamically orbiting satellites, drifting micro-asteroids, and a delivery bonus adding +20 s to the clock for each crystal secured. This is the ultimate mode for the ones who want to master the game.

(a) Testing the probe



(b) Running out of Time



(c) Spinning out of Space

Figure 4: Comparison of the three available operational game modes.

5 User Manual and Interaction Schemes

5.1 Keyboard Flight Controls

- **W / S:** Linear thrust forward / backward along the local Z-axis[c.
- **A / D:** Continuous roll left / right around the local Z-axis.
- **Up / Down (or Q / E):** Pitch up / down around the local X-axis.
- **Left / Right:** Yaw left / right around the local Y-axis.
- **P:** Deploy / fold solar panels (locks thrusters, activates recharging).
- **R:** Extend / fold robotic arm (activates precision speed damping).
- **G:** Grab / drop crystal via the end-effector claw.
- **C:** Toggle between third-person orbit view and first-person cockpit view.

5.2 GUI Debug Controls (lil-gui)

The floating GUI panel provides direct access to scene parameters:

- **CAMERA:** Toggles camera coordinate locks and perspective modes.
- **SOLAR ILLUMINATION:** Sliders for directional intensity, RGB color values, and Cartesian (X, Y, Z) coordinates of the main light source.
- **SPACE PROBE:** Manual triggers for arm folding, wing rotation, and claw attachment testing.

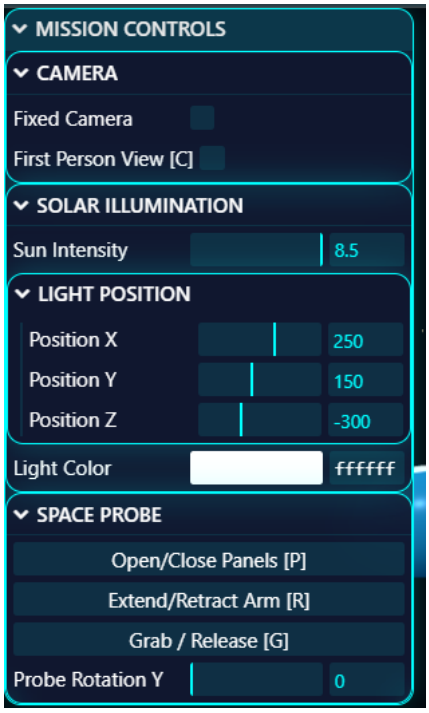


Figure 5: Control Display.

6 Technical Architecture and Mathematical Foundation

6.1 Hierarchical Scene Graph Structure

The space probe is modeled as a tree of nested transformation nodes. Transformations cascade down the graph via matrix multiplications:

$$M_{\text{world}} = M_{\text{root}} \times M_{\text{chassis}} \times M_{\text{joint}} \times M_{\text{link}}$$

The probe's hierarchical chain is organized as:

- **Probe Group (Root):** Coordinates global translation and orientation in world space.
  - **Chassis & Cockpit (Child):** Main hull mesh with customizable PBR parameters.
  - **Solar Wing Hinges (Child):** Bilateral nodes rotating around local X-axes during deployment.
  - **Robotic Arm Chain (Child):**

    Base Mount → Shoulder Joint → Upper Arm → Elbow Joint → Forearm → Claw Gripper

6.2 Dynamic Material Traversal and PBR Shaders

To apply user-selected hull finishes without altering engine thrusters or emissive flame meshes, a recursive scene-graph traversal algorithm is executed on the probe group:

- The hierarchy is traversed using `probeGroup.traverse()`.
- Valid chassis materials are updated with the selected hex color and roughness parameter, triggering `material.needsUpdate = true`.

Energy crystals use a multi-layer material model: an inner `MeshStandardMaterial` with metallic/roughness properties and an active `emissive` channel (emissiveIntensity = 0.6), enveloped by a slightly larger outer shell using `THREE.AdditiveBlending` and partial opacity ($\alpha = 0.25$) to simulate luminous energy diffusion.

6.3 Procedural Kinematic Animations

Rather than importing pre-made keyframes, all joint movements are evaluated at runtime using `Tween.js`. When the player triggers panel deployment or arm extension, joint angles interpolate according to quadratic easing functions:

$$\theta(t) = \theta_{\text{start}} + (\theta_{\text{end}} - \theta_{\text{start}}) \cdot f(t), \quad t \in [0, 1]$$

6.4 Energy Consumption Dynamics and Precision Control

Battery consumption is integrated per frame based on the active flight state:

$$E(t + \Delta t) = \max\Big(0,\ E(t) - \dot{E}_{\text{drain}} \cdot \Delta t\Big)$$

where the drain rate is set to:

$$\dot{E}_{\text{drain}} = \begin{cases} 1.2\,\%/s & \text{if thruster keys are pressed and panels are retracted} \\ 0.15\,\%/s & \text{in passive idle state (onboard electronics and life-support)} \end{cases}$$

When the robotic arm is extended for docking, an electronic safety damping factor $\kappa = 0.15$ is applied to both linear translational speed v and rotational rate ω:

$$\vec{v}_{\text{eff}} = \kappa \cdot v_{\text{base}}, \quad \vec{\omega}_{\text{eff}} = \kappa \cdot \omega_{\text{base}}, \quad \kappa = \begin{cases} 0.15 & \text{if arm is extended} \\ 1.0 & \text{if arm is folded} \end{cases}$$

This prevents sudden overshooting when aligning the claw with floating crystals.

Critical Power Threshold and Visual Alerts When remaining battery capacity drops below a critical safety threshold ($E(t) \leq 10\%$), the interface triggers an emergency state to alert the player:

- A pulsing red CSS vignette (`#danger-overlay`) is dynamically activated across the screen viewport via keyframe opacity animations.
- The HUD energy bar gradient shifts to high-visibility alert colors (`#ff3300` to `#ff0055`).
- If the battery reaches 0%, the probe loses all translation capability and triggers an immediate mission failure state.

6.5 Solar Recharging Vector Mechanics

To recharge, the player must open the solar panels and orient the probe's top face toward the Sun. Charging efficiency is computed using the vector dot product between the probe's top normal and the normalized sunlight direction vector:

1. Define the local unit top normal: $\vec{N}_{\text{local}} = (0, 1, 0)^T$.
2. Transform $\vec{N}_{\text{local}}$ to world coordinates using the probe's orientation quaternion Q_{probe}:

   $$\vec{N}_{\text{world}} = \mathbf{R}(Q_{\text{probe}}) \cdot \vec{N}_{\text{local}}$$

   where $\mathbf{R}(Q)$ is the 3×3 rotation matrix generated by Q_{probe}.
3. Compute the normalized sunlight vector from probe position $\vec{P}_{\text{probe}}$ to Sun position $\vec{P}_{\text{sun}}$:

   $$\vec{D}_{\text{sun}} = \frac{\vec{P}_{\text{sun}} - \vec{P}_{\text{probe}}}{\|\vec{P}_{\text{sun}} - \vec{P}_{\text{probe}}\|}$$

4. Calculate the alignment factor $A = \vec{N}_{\text{world}} \cdot \vec{D}_{\text{sun}} = \cos(\theta)$.

If $A > 0.82$ (angle within $\approx 35°$), charging activates, scaling linearly with alignment quality:

$$E_{\text{rate}} = E_{\max} \cdot \left(0.4 + 0.6 \cdot \frac{A - 0.82}{1.0 - 0.82}\right)$$

If $A \leq 0$, the panels face away from the light source, producing zero charge. Dynamic expressed through this snippet:

Listing 1: Vector Alignment and Dynamic Solar Recharge Calculation

```
const sunDir = new THREE.Vector3().subVectors(sunPos, probePos).normalize();
const topNormalLocal = new THREE.Vector3(0, 1, 0);
const topNormalWorld = topNormalLocal
  .applyQuaternion(spaceProbe.probeGroup.quaternion)
  .normalize();

const alignmentFactor = topNormalWorld.dot(sunDir);
const MIN_ALIGNMENT_THRESHOLD = 0.82;

if (alignmentFactor > MIN_ALIGNMENT_THRESHOLD) {
  const rechargeEfficiency = (alignmentFactor - MIN_ALIGNMENT_THRESHOLD) /
                             (1.0 - MIN_ALIGNMENT_THRESHOLD);
  const actualRecharge = MAX_RECHARGE_RATE *
                         (0.4 + 0.6 * rechargeEfficiency) * deltaTime;
  energy = Math.min(MAX_ENERGY, energy + actualRecharge);
  updateEnergyUI();
}
```

6.6 Orbital Dynamics and Satellite Trajectories

In *Spinning out of Space*, satellites follow planar circular orbits around the mothership origin. The position $\vec{P}_{\text{sat},i}(t)$ of each satellite is parameterized by its orbital radius R_i, angular velocity ω_i, vertical elevation Y_i, and initial phase ϕ_i:

$$\vec{P}_{\text{sat},i}(t) = \begin{pmatrix} R_i \cos(\omega_i t + \phi_i) \\ Y_i \\ R_i \sin(\omega_i t + \phi_i) \end{pmatrix}$$

Simultaneously, local axial tumbling is updated in the render loop: $\mathbf{R}_{\text{local}}(t) = \mathbf{R}_y(0.4t) \cdot \mathbf{R}_z(0.2t)$.

6.7 Procedural Spawning with Collision Avoidance

To ensure crystals do not spawn inside satellites or directly on top of the mothership, entity generation uses spatial rejection sampling. A candidate position $\vec{P}_{\text{candidate}}$ is accepted only if:

$$\|\vec{P}_{\text{candidate}} - \vec{P}_{\text{probe}}\| \geq 12.0 \quad \wedge \quad \min_j \|\vec{P}_{\text{candidate}} - \vec{P}_{\text{sat},j}\| \geq 8.5$$

6.8 Particle Starfield Synchronization

To maintain a dense cosmic background without geometry clipping against the far plane ($f = 3000$), the starfield particle system translates alongside the camera:

$$\vec{P}_{\text{stars}} = \vec{P}_{\text{cam}}$$

This keeps the camera at the exact center of the particle volume at all times, preserving visual depth.

6.9 First-Person View and Perspective Projections

In first-person mode, the camera attaches to a local cockpit offset $\vec{O}_{\text{local}} = (0, 0.8, -0.2)^T$:

$$\vec{P}_{\text{cam}} = \vec{P}_{\text{probe}} + (\mathbf{R}(Q_{\text{probe}}) \cdot \vec{O}_{\text{local}})$$
$$\vec{T}_{\text{forward}} = \vec{P}_{\text{probe}} + (\mathbf{R}(Q_{\text{probe}}) \cdot (0, -0.1, -10)^T)$$

The perspective projection matrix is recalculated with a wide field of view (FOV $= 85°$):

$$\mathbf{P}_{\text{proj}} = \begin{bmatrix} \frac{\cot(\text{FOV}/2)}{a} & 0 & 0 & 0 \\ 0 & \cot(\text{FOV}/2) & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

where a is the aspect ratio, n the near clipping plane, and f the far clipping plane.

6.10 Materials, Lighting, and Two-Tier Collision Pipeline

The scene lighting combines a primary `DirectionalLight` with ambient fill light. Objects use `MeshStandardMaterial` with configurable metalness and roughness values. Energy crystals combine PBR textures with emissive rendering (emissiveIntensity $= 0.6$) and an additive glow shell geometry (`AdditiveBlending`) to stand out against the dark space environment.

Celestial Environment and Thruster Particle FX The deep-space environment is enriched through multi-layered procedural celestial bodies and dynamic particle effects:

- **Sun Multi-Mesh Corona:** The main star is composed of three concentric geometries (an inner solid core, a semi-transparent corona shell, and an expansive outer glow using `THREE.AdditiveBlending` and `BackSide` rendering).
- **Background Planetary Bodies:** Four distinct planets (a ringed gas giant, an icy world, an active lava planet, and an atmospheric purple sphere) are placed in distant world coordinates and rotate on independent axial vectors.
- **Exhaust Thruster Dynamics:** Beyond the conical mesh flame, active forward acceleration spawns a buffer of 120 kinematic particles at the nozzle world coordinate, projecting them backward along the probe's inverted forward vector with dynamic velocity and lifetime decay.

Collisions are resolved through a two-tier pipeline:

- **Tier 1 (Broad-Phase Euclidean Proximity):** Radial distance checks evaluate proximity to crystals ($R = 1.25$), micro-asteroids ($R = 1.6$), and satellites ($R = 2.4$). Direct impacts with either satellites or drifting asteroids trigger fatal hull breach, instant battery depletion ($E = 0$), pronounced camera shake, and immediate mission failure.
- **Tier 2 (Narrow-Phase Bounding Box Intersection):** Collisions with the mothership evaluate actual bounding box intersections against the probe bounding sphere (`boundingBox.intersectsSphere`), reverting invalid positions to $\vec{P}_{\text{probe}}(t - \Delta t)$.

- **Object Capture & Delivery:** When the claw is within $d_{\text{claw}} < 1.8$ units, pressing **G** attaches the crystal as a child node of the gripper. Delivering it inside the cargo bay ($d < 6.0$ units) adds score points and grants a +20 s time extension in timed modes. This dynamic can be seen through this code snippet:

Listing 2: Two-Tier Collision Clamping and End-Effector Attachment Logic

```
// Narrow-phase collision clamping with Mothership hull
if (spaceProbe.boundingBox &&
    spaceProbe.boundingBox.intersectsSphere(cargoShip.boundingSphere)) {
  spaceProbe.probeGroup.position.copy(oldPosition);
}

// End-effector proximity capture
let minDistance = 1.8;
rocks.forEach(rock => {
  const dist = gripperPos.distanceTo(rock.position);
  if (dist < minDistance) {
    minDistance = dist;
    closestRock = rock;
  }
});
if (closestRock) spaceProbe.attachObject(closestRock);
```

- **Camera Shake Dynamics:** Physical impacts displace the camera and controls target using a decaying random offset vector:

$$\vec{P}_{\text{cam\_shake}} = \vec{P}_{\text{cam}} + \vec{\delta}, \quad \vec{\delta} \sim \mathcal{U}(-S/2, S/2)^3, \quad S(t + \Delta t) = \max(0, S(t) - 3.0\Delta t)$$

- **Dynamic Audio Tension:** When the mission countdown drops below 20 s, the soundtrack playback rate dynamically scales to 1.25×.

7 Conclusion and Future Work

Developing "Deep Space Cargo Recovery" allowed us to explore the core aspects of modern real-time 3D web graphics. Building the entire project from scratch with WebGL, Three.js, and Tween.js gave us direct control over every mechanic: from the articulated movements of the robotic arm and the vector math behind solar recharging, to the collision systems and dynamic PBR lighting.

7.1 Future Improvements

Looking ahead, there are several features we would like to explore to make the simulation even more complete:

- **Post-Processing Shaders:** Dynamic visual effects could be added, like a bloom glow for the crystals and motion blur when accelerating. We could use Three.js instanced meshes to fill the surrounding space with thousands of small, floating space rocks without dropping frame rates. Even add